ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

A PROJECT REPORT ON

MAJOR PROJECT

(ADVANCED JAVA - PROJECT BASED LEARNING)

SMART PG & HOSTEL MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirement for the award of Degree of

Bachelor of Technology in Artificial Intelligence & Data Science

Submitted to:

Rajasthan Technical University, Kota (Raj.)

Submitted By:

Dhruvraj Shekhawat	24EARAD045
Divyansh Paliwal	24EARAD047
Divyanshu Kumar	24EARAD048
Garvit Sharma	24EARAD050
Harsh Sharma	24EARAD057

Session: 2026-27

Department of Artificial Intelligence & Data Science

ARYA COLLEGE OF ENGINEERING & I.T.

SP-42, RIICO INDUSTRIAL AREA, KUKAS, JAIPUR

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence & Data Science

CERTIFICATE OF APPROVAL

The Project Report entitled **"SMART PG & HOSTEL MANAGEMENT SYSTEM"** submitted by "Dhruvraj Shekhawat (24EARAD045), Divyansh Paliwal (24EARAD047), Divyanshu Kumar (24EARAD048), Garvit Sharma (24EARAD050), Harsh Sharma (24EARAD057)" has been examined by us and is hereby approved for carrying out the project leading to the award of degree **"Bachelor of Technology in Artificial Intelligence & Data Science"**. By this approval the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn therein, but approve the pursuance of project only for the above mentioned purpose

Mentor

Er. Ram Babu Buri

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that the work embodies in this Project entitled **"SMART PG & HOSTEL MANAGEMENT SYSTEM"** being submitted by **"Dhruvraj Shekhawat (24EARAD045), Divyansh Paliwal (24EARAD047), Divyanshu Kumar (24EARAD048), Harsh Sharma (24EARAD057) and Garvit Sharma (24EARAD050)"** in partial fulfillment of the requirement for the award of **"Bachelor of Technology in Artificial Intelligence & Data Science"** to Rajasthan Technical University, Kota (Raj.) during the academic year 2026-27 is a record of bonafide piece of work, carried out by them under our supervision and guidance in the **"Department of Artificial Intelligence & Data Science", Arya College of Engineering & Information Technology, Jaipur.**

Mentor	**HOD**
Er. Ram Babu Buri	**Dr. Ashok Kumar Kajla**
Assistant Professor	**Professor**

ARYA College of Engineering & Information Technology
(Approved by AICTE & Affiliated to RTU, Kota)
SP-42, RIICO Industrial Area, Delhi Road, Kukas, Jaipur - 302028 (Rajasthan) India
Tel. No. : 0141-6604555
Toll Free : 1800-266-2000
Email Id : info@aryacollege.in
Website : www.aryacollege.in
Estd. Yr. 2000
ACCREDITED BY: NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL | NBA NATIONAL BOARD OF ACCREDITATION FOR ECE & IT

Department of Artificial Intelligence & Data Science

DECLARATION

We **"Dhruvraj Shekhawat, Divyansh Paliwal, Divyanshu Kumar, Harsh Sharma, and Garvit Sharma"**, students of Bachelor **of Technology** in **Artificial Intelligence & Data Science, session 2026-27, Arya College of Engineering & Information Technology, Jaipur**, hereby declare that the work presented in this Project entitled **" SMART PG & HOSTEL MANAGEMENT SYSTEM"** is the outcome of our own work, is bonafide and correct to the best of our knowledge and this work has been carried out taking care of Engineering Ethics. The work presented does not infringe any patented work and has not been submitted to any other University or anywhere else for the award of any degree or any professional diploma.

Dhruvraj Shekhawat (24EARAD045)
Divyansh Paliwal (24EARAD047)
Divyanshu Kumar (24EARAD048)
Harsh Sharma (24EARAD057)
Garvit Sharma (24EARAD050)

Date : _____

TABLE OF CONTENT

1. INTRODUCTION 1
2. OBJECTIVES 2
3. SOFTWARE & TECHNOLOGY DESCRIPTION 3
4. SYSTEM ARCHITECTURE & DATABASE DESIGN 4
5. WORKING PRINCIPLE 7
6. METHODOLOGY 9
7. FLOWCHARTS & SYSTEM DIAGRAMS 11
8. RESULT & DISCUSSION 14
9. PERFORMANCE OBSERVATIONS 17
10. CONCLUSION AND RECOMMENDATION 18
11. FUTURE SCOPE 19
12. TEAM CONTRIBUTION 20
13. REFERENCES 22
14. APPENDICES 23

- PROJECT LINKS
- PROJECT CODE
- REST API CATALOG

1. Introduction

In today's rapidly growing education sector, thousands of students move away from their homes every year to study in colleges and universities. Hostels and Paying Guest (PG) accommodations have therefore become an essential part of the campus ecosystem. Managing such a residence is a complex job. The administration has to maintain student records, allot rooms and beds, collect monthly rent and mess charges, handle maintenance problems, approve leave and visitors, publish notices and plan the weekly mess menu. In many hostels these activities are still carried out with paper registers, spreadsheets and verbal communication, which are slow, error-prone and difficult to audit.

The **Smart PG & Hostel Management System** is a web-based enterprise application developed to digitise all these day-to-day operations on a single secure platform. The system is built using **Java 17**, **Spring Boot 3.3.4**, **Spring MVC**, **Spring Security 6**, **Spring Data JPA (Hibernate 6)**, **Thymeleaf**, **Bootstrap 5** and a **MySQL 8** relational database hosted on the Aiven cloud. The application follows a layered architecture (Controller, Service, Repository and Entity layers) and is deployed on the Render cloud platform using a Docker container, so that it can be accessed from any browser without installing anything on the client side.

The application is designed around four user roles. The **Warden** acts as the administrator and controls students, rooms, allocations, notices, mess menu, leave and visitor requests and the final verification of complaints. The **Student** (resident) can view the allotted room, pay hostel fees, lodge complaints, request leave passes and visitor passes and read notices. The **Accountant** generates monthly fee invoices, records payments and views collection reports. The **Maintenance (Complaint) Staff** receives complaint tickets automatically, accepts them, performs the repair work and submits the resolution for the warden's verification. Access to every page and every action is protected by role-based authorisation.

Problem Statement:

Manual hostel administration suffers from duplicate data entry, lack of real-time room availability, delayed fee collection, no tracking of complaints, uncontrolled visitor entry and the absence of a transparent history of who approved what. Students do not have any single place where they can see their room, dues and request status. The project addresses these problems by providing one integrated, role-based and auditable web platform.

Applications:

- College and university boys / girls hostels
- Private Paying Guest (PG) accommodations and co-living spaces
- Working-professional hostels and training-institute residences

2. Objectives:

- To design and implement a secure, role-based web application for complete PG and hostel management using Spring Boot and MySQL.
- To maintain digital records of students, rooms, bed capacity and room allocations and to prevent over-allocation of beds.
- To automate monthly fee invoicing, online and counter payments, pending-balance calculation and fee status tracking (Pending, Partial, Paid, Overdue).
- To provide a transparent complaint workflow with automatic routing to the maintenance department, SLA deadlines, warden verification and student feedback.
- To digitise leave passes and visitor gate passes with approval, rejection, check-in and check-out tracking.
- To implement authentication and authorisation using Spring Security, HttpSession and BCrypt password hashing for four different roles.
- To publish notices and the weekly mess timetable and to give every role a dashboard with real-time statistics.
- To generate reports and PDF receipts and to keep an audit trail of important actions performed in the system.
- To apply layered architecture, JPA relationships, validation and global exception handling in a practical enterprise-style project.
- To deploy the application on the cloud (Render + Aiven MySQL) using Docker and environment variables so that it is accessible online.

Scope of the Project:

The scope of the system covers one or more hostel blocks managed by a single administration. It includes student registration and profile management, room and bed management, allocation workflow, fee and payment management, complaints, visitor and leave management, notices, mess menu, dashboards and reports. Features such as biometric attendance, IoT based smart locks, biometric entry and integration with a real payment gateway are kept outside the present scope and are suggested as future enhancements.

3. Software & Technology Description

The project is built entirely with open-source Java technologies. The table below summarises each technology, its role in the system and the way it is used in the project.

Technology	Description and Use in the Project
Java 17 (JDK)	Platform-independent, object-oriented language and the LTS baseline required by Spring Boot 3. The complete backend - controllers, services, repositories, entities and security - is written in Java.
Spring Boot 3.3.4	Provides auto-configuration, embedded Tomcat, starter dependencies and externalised configuration. Runs the application on port 2007 (PORT variable) and packages it as one executable JAR.
Spring MVC + Thymeleaf	MVC controllers return Thymeleaf views. The project has 55 server-rendered templates with reusable fragments (navbar, sidebar, alerts) and role-aware menus through the Spring Security dialect.
Spring Security 6	Form login, HttpSession (SMART_HOSTEL_SESSION, 60 min), BCrypt hashing, role rules such as hasRole("WARDEN") for /warden/**, custom access-denied handler and logout. A JWT filter secures the REST API under /api/**.
Spring Data JPA + Hibernate 6	ORM layer with 14 entities and 14 repositories, relationships (@OneToOne, @ManyToOne, @OneToMany), enums, JPQL queries and @Transactional service methods.
MySQL 8 (Aiven Cloud)	Relational database connected through JDBC with SSL. Credentials are read from DB_URL, DB_USER and DB_PASSWORD; tables are created by Hibernate (ddl-auto=update).
Bootstrap 5 + Icons	Responsive UI: grid, cards, tables, badges, forms and alerts, with a custom dark sidebar and a Dark Mode toggle.
Maven, Lombok, Validation	Maven builds and manages dependencies; Lombok removes boilerplate; Jakarta Bean Validation (@NotBlank, @Email, @Size) validates every request on the server side.
JJWT 0.12.6, OpenPDF 2.0.3	JJWT issues HMAC-SHA256 signed tokens for REST calls; OpenPDF generates fee receipts and financial / occupancy PDF reports.
React 18 + Vite (optional)	A decoupled single-page client in the repository that consumes the same REST API.
Docker, Render, GitHub, Postman	Multi-stage Dockerfile (Maven + JDK 17 to JRE 17), cloud deployment on Render, source control on GitHub and a Postman collection for API testing.

System Requirements:

Category	Development Environment	End-User / Client Side
Processor	Intel Core i3 or higher	Any modern processor
RAM	8 GB recommended (4 GB minimum)	2 GB or more
Operating System	Windows 10/11, Linux or macOS	Windows, Android, iOS, Linux
Software	JDK 17, Maven, IntelliJ IDEA / VS Code, Git, MySQL; internet access	Chrome, Edge, Firefox or Safari with internet

4. System Architecture & Database Design

The system follows a classic **layered (N-tier) architecture** that separates presentation, request handling, business logic and data access. A request from the browser first passes through the Spring Security filter chain, which checks authentication and the role of the user. The request is then mapped to a controller, the controller calls a service, the service applies the business rules inside a transaction and uses repositories to read or write data through JPA/Hibernate in the MySQL database. The response is finally rendered by a Thymeleaf template and returned to the browser.

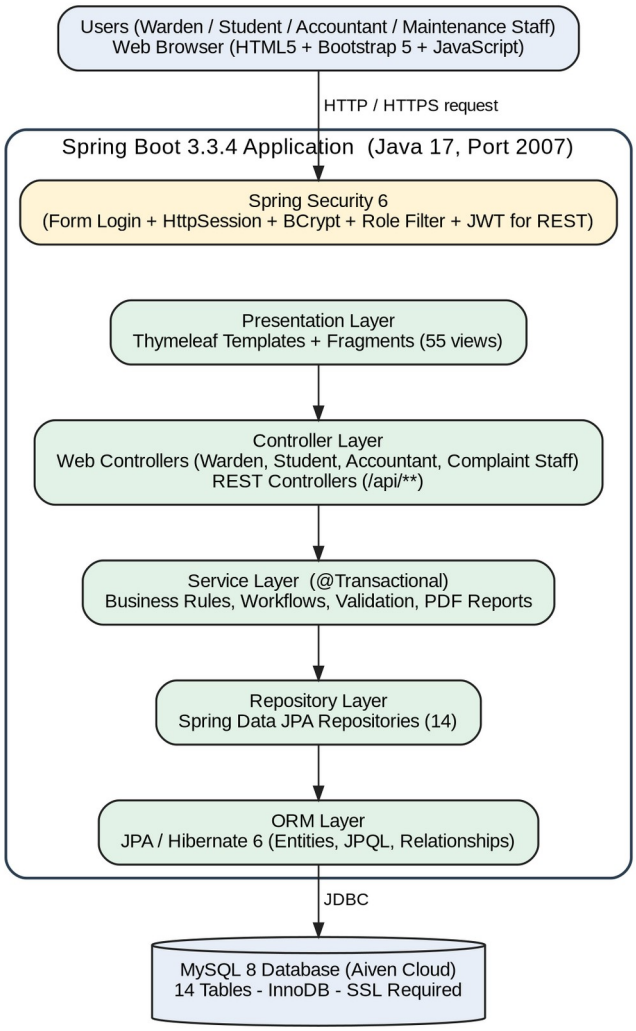


Fig. 4.1: Layered Architecture of the System

Database Configuration:

Important runtime settings are listed below; sensitive values are read from environment variables.

Parameter	Value / Source	Description
server.port	PORT (default 2007)	HTTP port of the embedded server
spring.datasource.url / user / password	DB_URL, DB_USER, DB_PASSWORD	Aiven MySQL (SSL) connection, secrets from environment
spring.jpa.hibernate.ddl-auto	update	Creates / updates tables automatically
server.servlet.session.timeout	60 minutes	Idle session expiry
app.jwt.expiration-ms	86400000 (24 hours)	Validity of REST API token

Database Tables:

The database is normalised up to the third normal form with auto-generated primary keys and foreign-key relationships. The 14 tables are listed below.

Table	Purpose	Important Columns
users	Login accounts of all four roles	name, email (unique), password (BCrypt), role, enabled
students	Resident profile linked 1:1 with user	admission_number, college, course, guardian details
staff	Warden, accountant and maintenance staff details	employee_id, department, designation
rooms	Rooms with bed capacity and rent	room_number, room_type, capacity, occupied, rent_per_month, status
room_allocations	Request, approval and history of bed allotment	student_id, room_id, bed_number, status, start_date, end_date
fees	Monthly invoice per student (unique per month)	room_rent, mess_fee, electricity_fee, maintenance_fee, total, paid, remaining, status
payments	Every payment transaction	transaction_id, amount, payment_method, payment_status
complaints	Maintenance tickets with SLA and verification data	category, priority, status, sla_deadline, resolution_remarks
leave_requests	Leave passes with approval trail	leave_type, from_date, to_date, status, admin_remarks
visitor_requests	Visitor gate passes	visitor_name, visit_date, check_in_time, check_out_time, status
notices	Announcements with priority and expiry	title, content, priority, target_audience, expiry_date
mess_menus	Weekly mess timetable (unique per day and meal)	day_of_week, meal_type, items, timing
notifications	In-app alerts for complaint events	recipient_role, title, message, is_read
audit_logs	Audit trail of important actions	user_name, user_role, action, module, record_id

Entity Relationship Diagram:

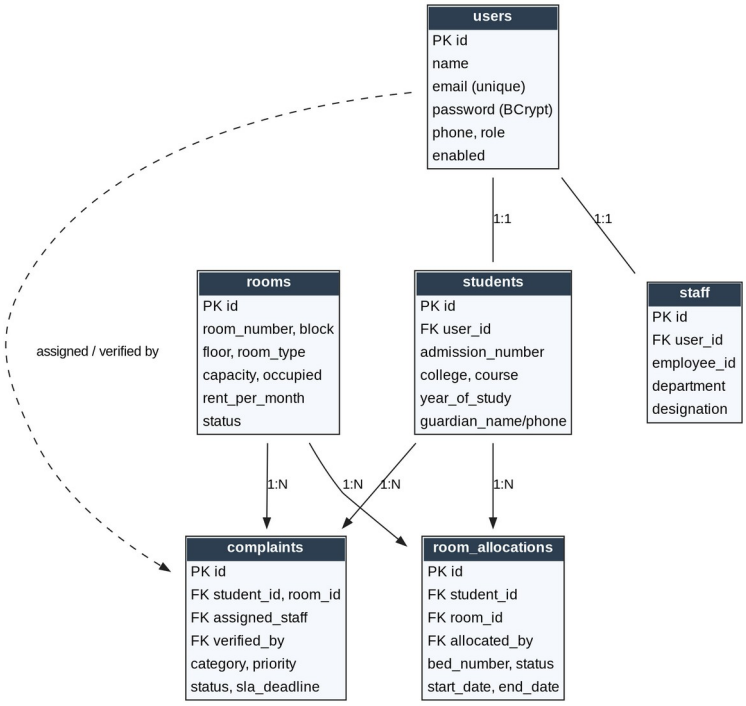


Fig. 4.2: ER Diagram - Core Entities (Users, Students, Rooms, Allocation, Complaints)

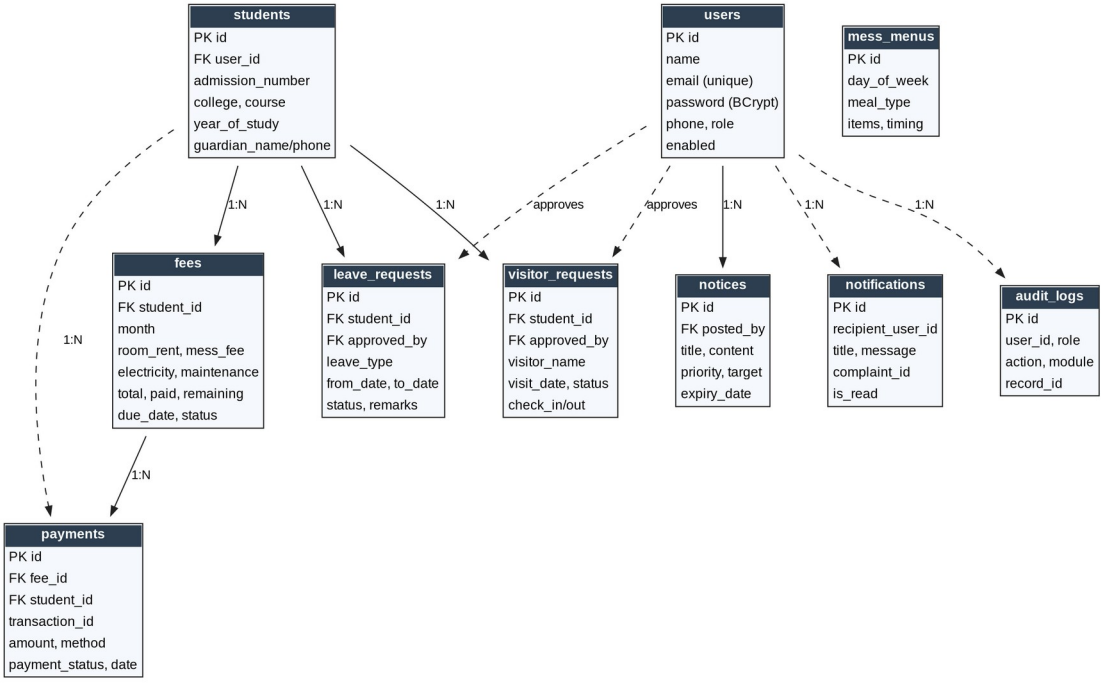


Fig. 4.3: ER Diagram - Finance, Request and Service Entities

5. Working Principle

The Smart PG & Hostel Management System works on the principle of **role-based workflow automation**. Every business process of the hostel - room allotment, fee collection, complaint handling, leave and visitor approval - is modelled as a sequence of states. A user with the proper role moves a record from one state to the next, the service layer validates the rule, the change is stored inside a database transaction and the other role immediately sees the result on the dashboard.

Step 1 - Authentication: The user enters e-mail and password on /login. Spring Security loads the account and compares the password with the stored BCrypt hash; on success an HttpSession is created, otherwise the user returns to the login page and no session is created.

Step 2 - Role-based Redirection: The success handler redirects the user to /warden, /student, /accountant or /complaint-staff dashboard. Every later request is checked against URL rules such as hasRole("WARDEN"); a student opening a warden URL gets the Access Denied page.

Step 3 - Student and Room Management: The warden registers students (user account plus student profile with BCrypt password) and maintains rooms with block, floor, type, capacity, rent and status. Bean Validation checks all inputs.

Step 4 - Room Allocation: On approval the service checks that the student has no ACTIVE allocation, the room is not under maintenance and a bed is free; it then increments the occupied count, updates the room status (AVAILABLE or FULL), stores the bed number and marks the allocation ACTIVE. Vacating decrements the count.

Step 5 - Fee Invoicing and Payment: The accountant generates one invoice per student per month (rent + mess + electricity + maintenance); the total is calculated and the status is PENDING. A student pays online or the accountant records a counter payment; the service blocks over-payment and payment of another student's invoice, generates a unique transaction ID and recalculates paid, remaining and status (PARTIAL or PAID).

Step 6 - Complaint Workflow: A student lodges a complaint; it is routed automatically to the Maintenance Department with status SUBMITTED and an SLA deadline (4, 8, 24 or 48 hours by priority). Staff accept it (IN_PROGRESS) and resolve it with remarks (RESOLVED_BY_MAINTENANCE); the warden verifies (VERIFIED) or returns it (RETURNED), and after student feedback it is CLOSED.

Step 7 - Leave and Visitor Passes: The student submits a leave request (type, dates, reason) or pre-registers a visitor. Dates are validated, the request is saved as PENDING and the warden approves or rejects it with remarks; visitor check-in and check-out complete the pass.

Step 8 - Notices, Mess Menu, Dashboards and Reports: The warden manages notices and the 7-day mess menu, which students read on their dashboard. Each role dashboard is built from live database counts, and PDF receipts and reports are produced with OpenPDF.

Step 9 - Audit and Notification: Important actions are written to audit_logs (user, role, module, record ID) and complaint events create notifications for the concerned role.

Role-wise Functional Matrix:

Module / Function	Warden	Student	Accountant	Maint. Staff
Login / Logout / Dashboard	Yes	Yes	Yes	Yes
Student & Room CRUD	Yes	View own	-	-
Room Allocation	Approve / Vacate	Request	-	-
Fee Invoices	View	View own	Create / Edit	-
Payments & Fee Reports	View	Pay online	Record / Report	-
Complaints	Verify / Return	Lodge / Feedback	-	Accept / Resolve
Leave Requests	Approve / Reject	Apply	-	-
Visitor Passes	Approve / Check-in	Request	-	-
Notices & Mess Menu	Create / Edit	Read	-	-
User Account Management	Yes	-	-	-

6. Methodology

The project was developed by a five-member team (Team Nexus) under the guidance of the project mentor, following a structured 10-week Project Based Learning plan that covers requirement analysis, design, coding, integration, testing, deployment and documentation. An agile, module-wise approach was adopted so that every member owned one primary module while taking part in integration and testing.

i. Team Formation, SRS and Design:

The team formed, finalised the problem statement, created the GitHub repository and prepared the abstract and the Software Requirement Specification (functional and non-functional requirements for the four roles). Use case, class, activity and sequence diagrams, the ER diagram with 14 tables and Bootstrap UI mock-ups were then designed.

ii. Module-wise Implementation:

Coding was done week by week: Authentication, Student and Room; then Allocation, Complaints and Fees; then Payments, Visitors and Leave; and finally Notices, Mess Menu, Reports and Dashboards. Each module follows the pattern Entity, Repository, Service, Controller and Thymeleaf view, data-changing service methods use @Transactional, and Spring Security (form login, BCrypt, role rules) plus a global exception handler were added.

iii. Integration and Testing:

After module testing, the complete application was tested end to end for all four roles (login, access control, allocation limits, fee calculation, complaint lifecycle, leave validation, visitor flow and error handling), defects were fixed and the stable build was merged into the main branch.

iv. Deployment and Documentation:

The application was containerised with a multi-stage Dockerfile and deployed on Render with the database on Aiven MySQL; credentials and the JWT secret are supplied as environment variables and DataInitializer seeds rooms, demo accounts and the mess menu. The report, PPT and demo video were prepared in the last week.

Ten-Week Execution Plan:

Week	Official Milestone	Project Focus
1	Team formation, guide selection, abstract	Scope, abstract, roles, repository
2	SRS	Software requirement specification
3	UML design	Use case, class, activity and sequence diagrams
4	Database design and UI mock-ups	ER diagram, schema, Bootstrap mock-ups
5	Coding: Authentication, Student, Room	Login, session, student and room CRUD
6	Coding: Allocation, Complaints, Fees	Allocation rules, complaint flow, fee logic
7	Coding: Payments, Visitors, Leave	Payments, visitor and leave workflows
8	Coding: Notices, Mess, Reports, Dashboards	Announcements, mess menu, reports, dashboards
9	Integration and testing	End-to-end testing and bug fixing
10	Report, PPT, demo video, final viva	Documentation, deployment and presentation

7. Flowcharts & System Diagrams

This chapter presents the Use Case diagram, the Data Flow Diagrams, the flowcharts of the two main workflows and the deployment diagram of the Smart PG & Hostel Management System. They were prepared in the design phase and describe how the four actors interact with the system.

7.1 Use Case Diagram:

The diagram shows the four actors - Warden, Student, Accountant and Maintenance Staff - and the functions available to each. Login and logout are common to all actors.

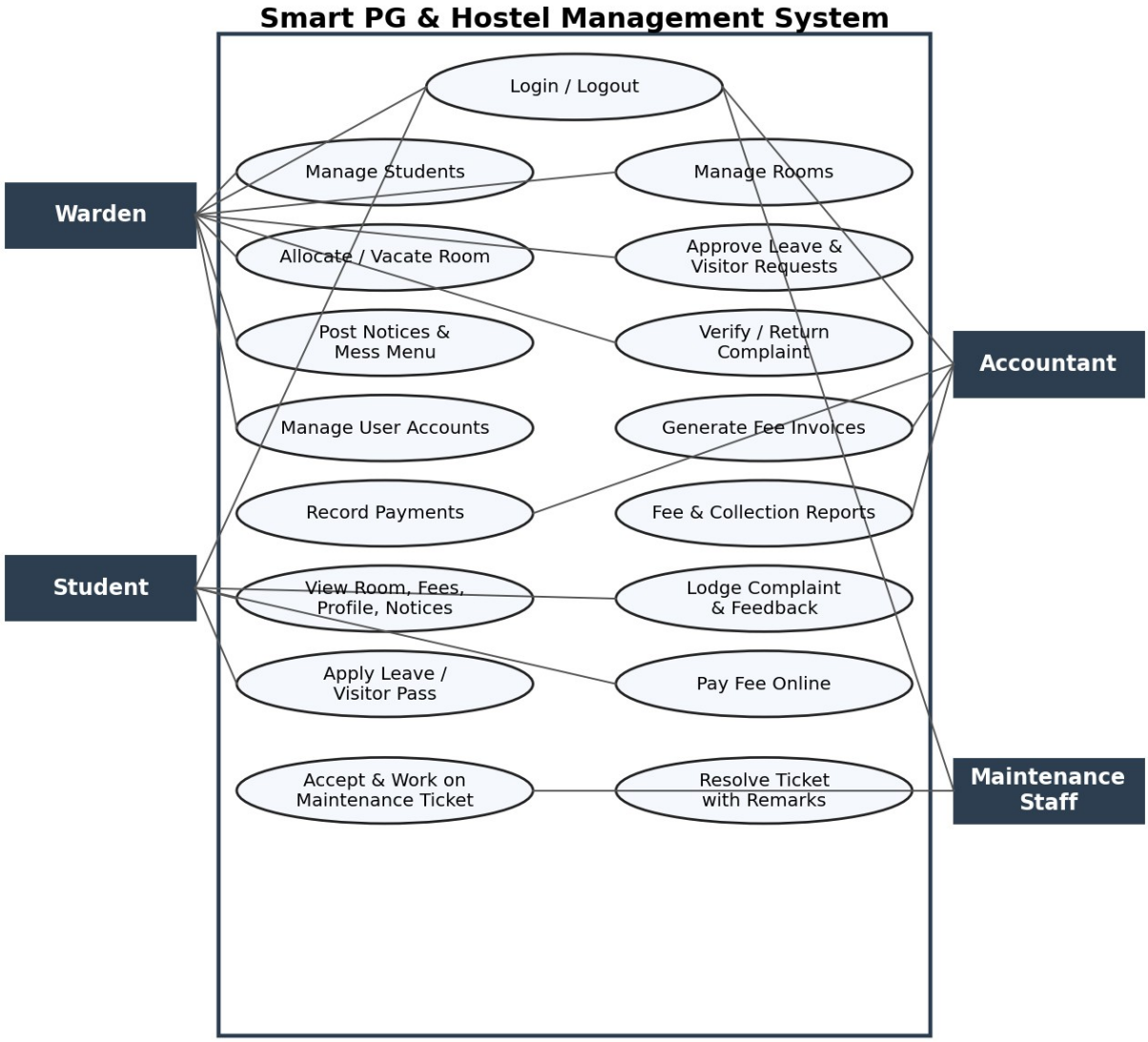


Fig. 7.1: Use Case Diagram

7.2 Data Flow Diagrams and Deployment:

The Level-0 diagram represents the whole system as one process exchanging data with four external entities; the Level-1 diagram decomposes it into seven processes and their data stores. The deployment diagram shows how the code reaches the cloud.

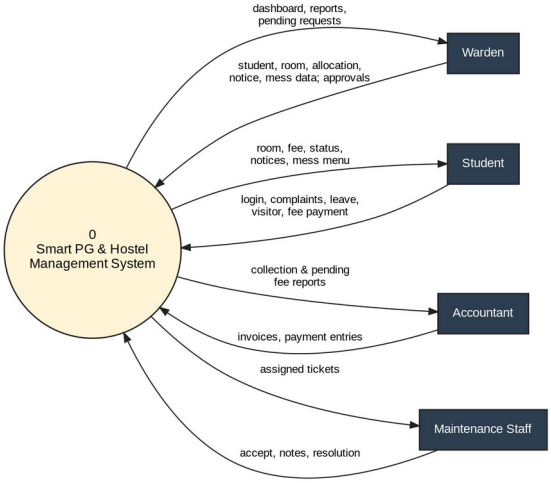


Fig. 7.2: Level-0 DFD (Context Diagram)

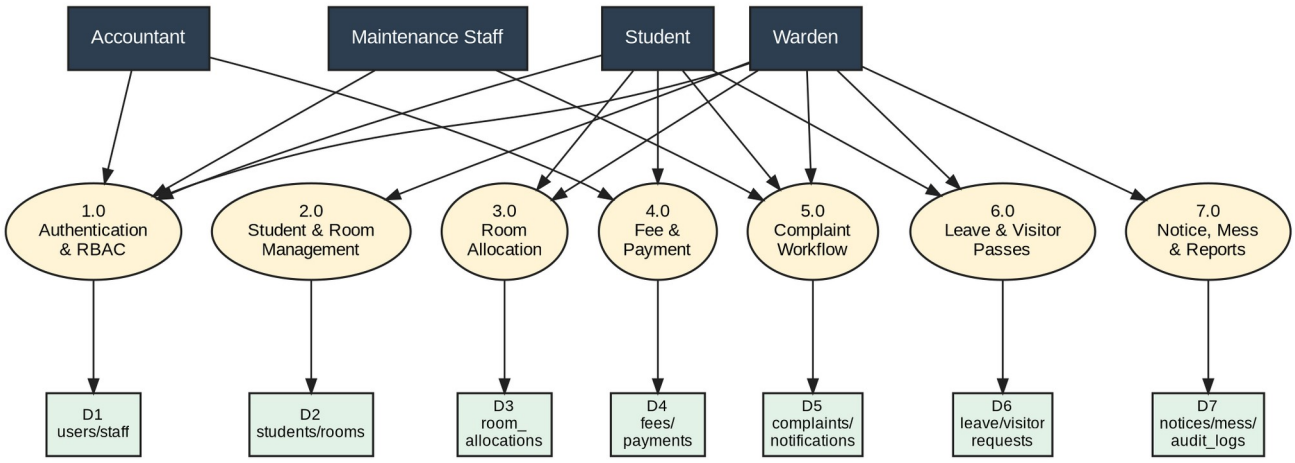


Fig. 7.3: Level-1 DFD

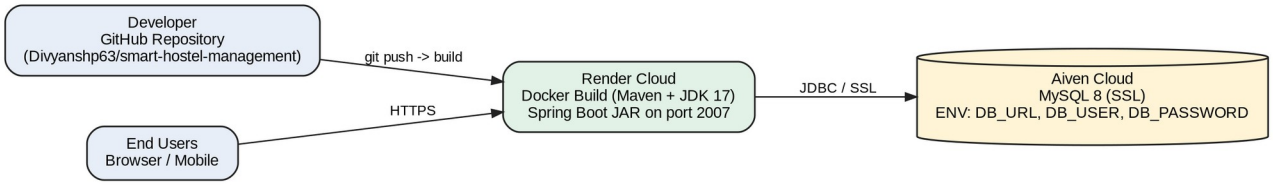


Fig. 7.4: Deployment Diagram

7.3 Flowcharts - Room Allocation and Complaint Lifecycle:

The left flowchart shows the rules enforced before a bed is assigned and the occupied-count update; the right flowchart shows the complaint ticket from submission to closure, including the rework loop.

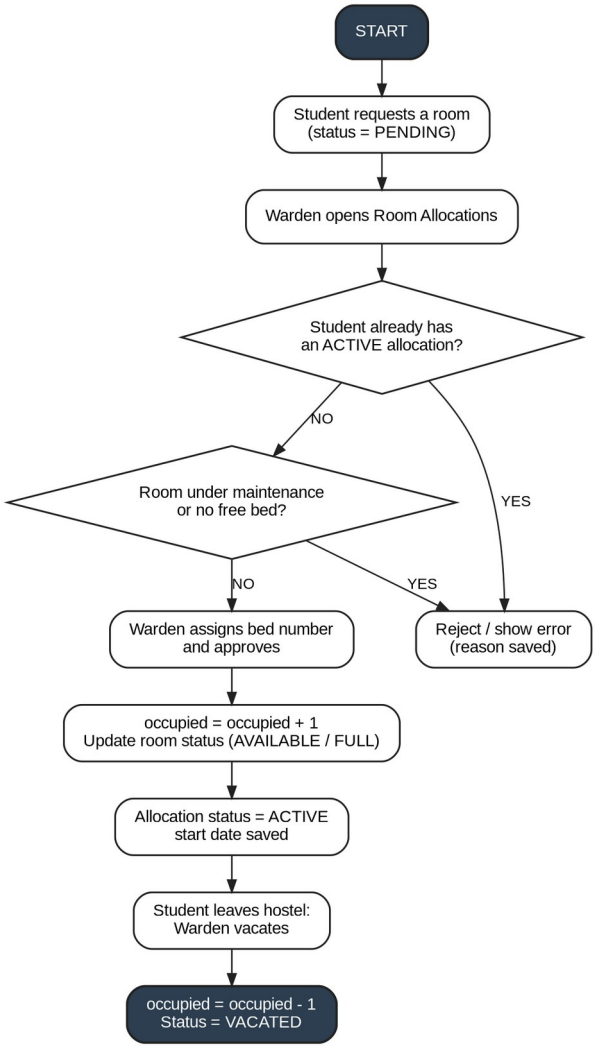


Fig. 7.5: Room Allocation Flowchart

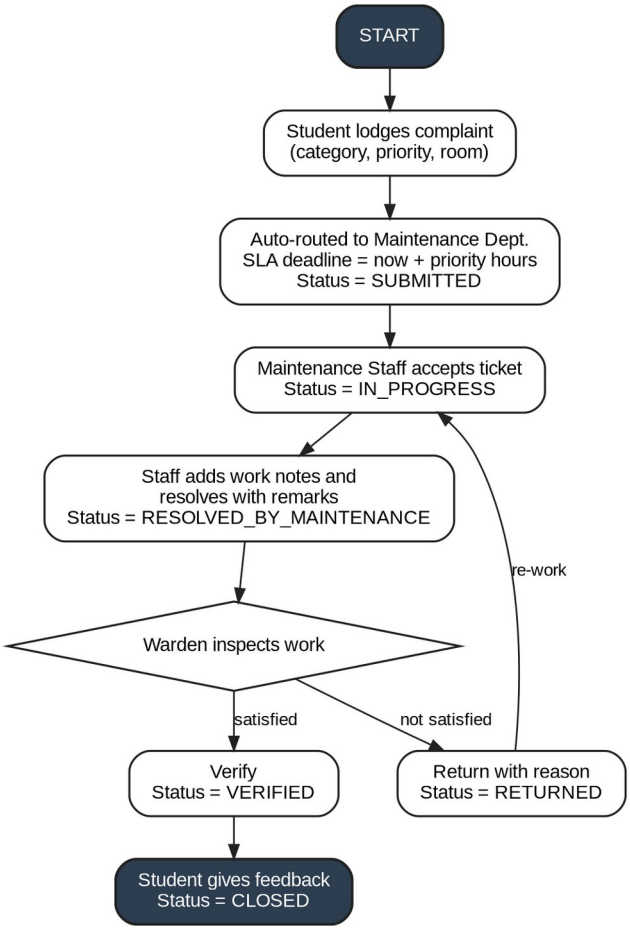


Fig. 7.6: Complaint Lifecycle Flowchart

8. Result & Discussion

The Smart PG & Hostel Management System was tested extensively on the deployed application and on the local environment for all four roles. The functional behaviour, access control, data integrity and workflow correctness were verified. This chapter shows the working screens of the four role modules, followed by the test-result table and a discussion of the results.

8.1 Warden Module - Hostel Operations Dashboard:

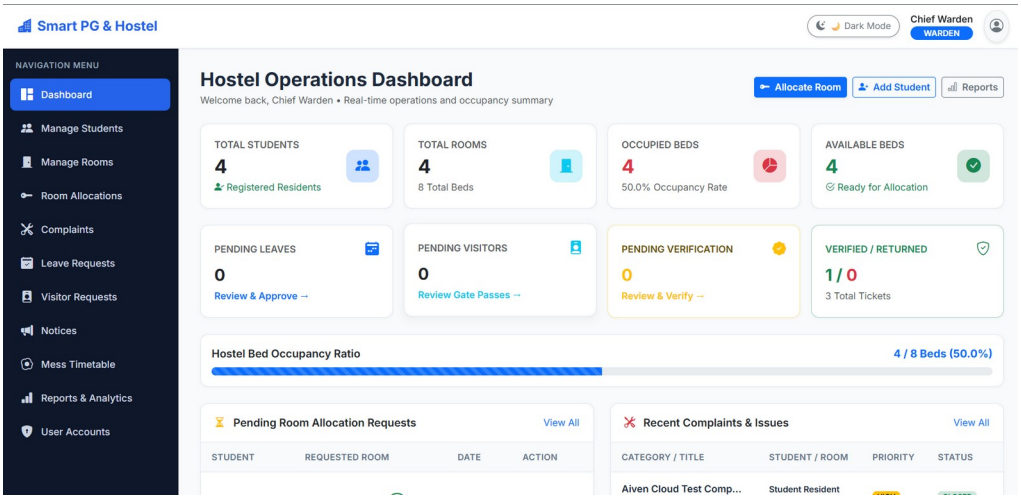


Fig. 8.1: Warden Dashboard

The dashboard shows registered students, rooms, occupied and available beds (4 of 8 beds, 50% occupancy), pending leave and visitor requests and complaints waiting for verification, with quick actions Allocate Room, Add Student and Reports.

8.2 Maintenance Staff Module - Complaint Queue:

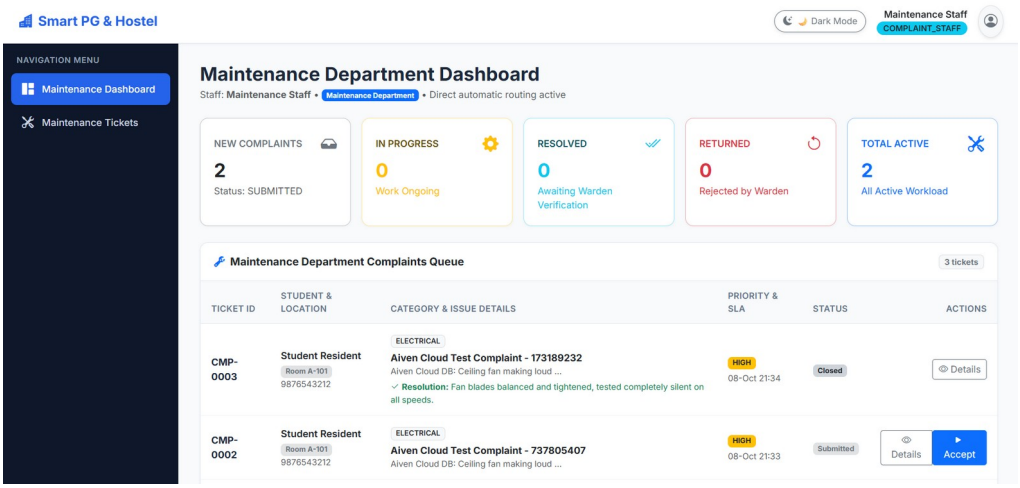


Fig. 8.2: Maintenance Department Dashboard

Complaints from students are routed automatically to this queue with ticket ID, room, category, priority, status and action buttons. The cards show new, in-progress, resolved and returned tickets; a closed ticket with its resolution remark and a new ticket with the Accept button are visible.

8.3 Student Module - Resident Portal:

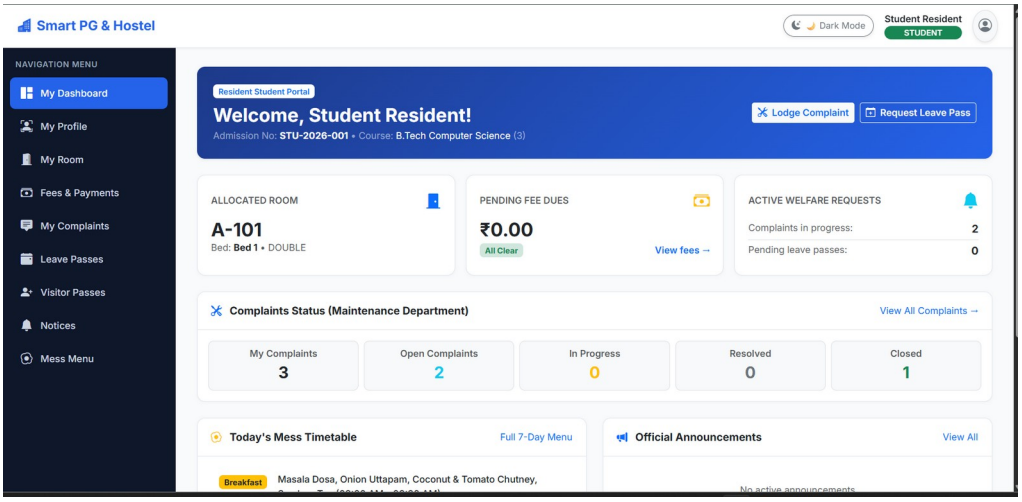


Fig. 8.3: Student Dashboard

The portal shows the admission number and course, the allocated room (A-101, Bed 1, double sharing), pending fee dues (All Clear), a summary of the student's complaints, today's mess timetable and announcements, with shortcuts to Lodge Complaint and Request Leave Pass.

8.4 Accountant Module - Financial Dashboard:

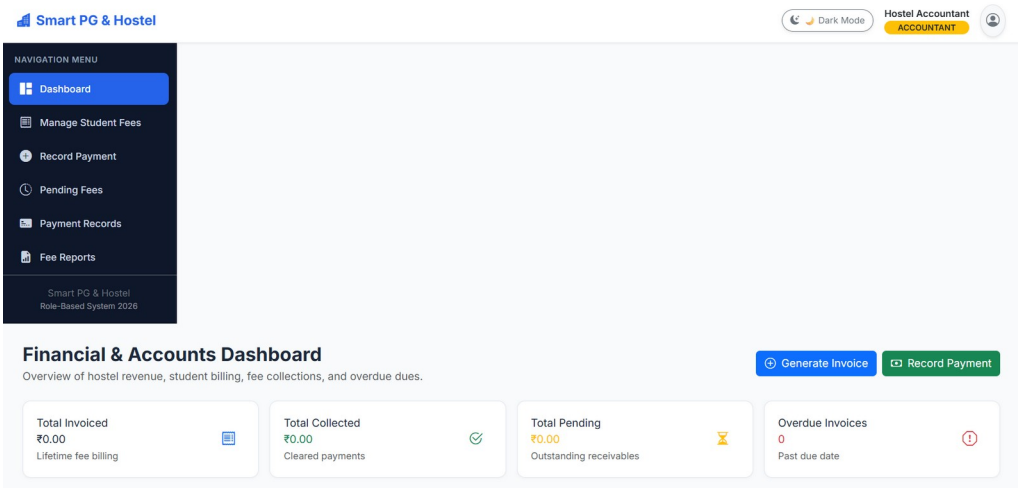


Fig. 8.4: Accountant Dashboard

The dashboard shows total invoiced, collected and pending amounts and overdue invoices, with buttons Generate Invoice and Record Payment. In the captured demo database no invoice had been generated yet, so all amounts are zero.

8.5 Test Results Table:

Test No.	Module	Test Scenario / Input	Observed Behaviour	Result
1	Login	Valid warden credentials	Redirected to /warden/dashboard	PASS
2	Login	Valid student credentials	Redirected to /student/dashboard	PASS
3	Login	Valid accountant credentials	Redirected to /accountant/dashboard	PASS
4	Login	Valid maintenance staff credentials	Redirected to /complaint-staff/dashboard	PASS
5	Login	Wrong password	Error message shown, no session created	PASS
6	Security	Student opens a warden URL	Access Denied page shown	PASS
7	Logout	Click Logout	Session invalidated, login page with logout message	PASS
8	Room	Add room with capacity and rent	Room saved and listed in Manage Rooms	PASS
9	Student	Add student with valid data	User and student profile created	PASS
10	Allocation	Approve request with free bed	Occupied count +1, allocation ACTIVE	PASS
11	Allocation	Allocate in full / maintenance room	Request rejected with message	PASS
12	Fee	Generate invoice with four components	Total computed, status PENDING	PASS
13	Payment	Partial payment	Status PARTIAL, remaining amount correct	PASS
14	Payment	Pay remaining amount	Status PAID, transaction ID generated	PASS
15	Payment	Pay on a PAID invoice / over-payment	Rejected with message	PASS
16	Complaint	Student lodges high-priority complaint	Ticket created, SUBMITTED, SLA 8 h, in staff queue	PASS
17	Complaint	Staff accepts and resolves ticket	IN_PROGRESS then RESOLVED_BY_MAINTENANCE	PASS
18	Complaint	Warden verifies / returns ticket	VERIFIED / RETURNED correctly	PASS
19	Complaint	Student gives feedback	Ticket status CLOSED	PASS
20	Leave	From-date later than to-date	Validation error shown	PASS
21	Leave	Warden approves leave	Status APPROVED with approver and time	PASS
22	Visitor	Visitor request, approval, check-in/out	PENDING, APPROVED, COMPLETED	PASS
23	Notice / Mess	Create notice, edit mess item	Visible to students immediately	PASS
24	Errors	Open an invalid URL	Custom 404 page shown	PASS

8.6 Discussion:

All 24 functional test scenarios produced the expected behaviour (100% pass rate). The system correctly enforces authentication, role-based access, allocation capacity limits, fee calculation rules and the multi-step complaint workflow, and the dashboard values match the data updated during allocation, payment and complaint handling. Automatic routing with SLA deadlines and the two-step closure (maintenance resolution, then warden verification and student feedback) give accountability that a paper register cannot, while @Transactional service methods keep occupancy and fee balances consistent.

9. Performance Observations

Application Responsiveness:

Server-side rendered pages load quickly because each request performs a few indexed look-ups through Spring Data JPA and the CSS and JavaScript files are static. Navigation felt smooth for all four roles on desktop and mobile screens. On free cloud instances the first request after a long idle period may take longer while the container restarts.

Data Integrity and Transaction Safety:

Allocation, vacating, payment and complaint-status changes run inside @Transactional methods, so a failed step rolls back the whole operation. Unique constraints (one invoice per student per month, one mess entry per day and meal, unique e-mail) prevent duplicates, and BigDecimal is used for money values.

Security Behaviour:

Passwords are stored only as BCrypt hashes, sessions expire after 60 minutes of inactivity, logout invalidates the session and every URL group is guarded by a role rule. Ownership checks stop a student from paying another student's invoice. During testing no role could open the pages of another role.

Workflow Accuracy:

SLA hours (4, 8, 24 and 48) are assigned automatically from the complaint priority, complaint states moved in the correct order and invalid transitions were rejected by the service layer.

Usability and Error Handling:

A common layout with a dark sidebar, status badges, statistic cards and a Dark Mode switch gives a consistent look. Bootstrap alerts and field-level validation messages guide the user, and a GlobalExceptionHandler with custom 403, 404 and 500 pages hides technical details.

Deployment, Scalability and Maintainability:

The Docker image builds with Maven and runs on a small Java 17 JRE; SSL database access through environment variables lets the same image move between environments. The layered structure and the REST API next to the MVC pages make it easy to add new modules.

10. Conclusion and Recommendation

The Smart PG & Hostel Management System project successfully demonstrates the use of Advanced Java technologies to solve a real-world administration problem. The system provides a secure, role-based, cloud-deployed web application in which the warden, students, accountant and maintenance staff work on a single platform with real-time information.

The project shows how automation can improve the efficiency, transparency and convenience of hostel management. Digital room allocation with automatic occupancy updates removes double booking; monthly invoicing with partial payment support and status tracking improves fee collection; and the complaint workflow with automatic routing, SLA deadlines, verification and feedback ensures that student problems are not ignored. Leave and visitor passes bring discipline and security to the hostel gate.

However, certain challenges such as the dependency on internet and cloud services, the recurring cost of hosting, regular database backup and the maintenance of software updates must be considered. Despite these limitations the benefits of the system clearly outweigh the drawbacks, making it a reliable and effective solution.

Recommendations:

- Change all default demonstration passwords and the JWT secret before using the system in a real hostel.
- Keep database credentials only in environment variables and take regular automated backups of the MySQL database.
- Enable HTTPS only access and enable CSRF protection for the form based pages in production deployment.
- Train the warden, accountant and maintenance staff with a short demonstration before going live.
- Integrate a licensed payment gateway before collecting real online payments.

In conclusion, the Smart PG & Hostel Management System is a practical application of Java, Spring Boot and database technologies that addresses real problems efficiently. It contributes to smarter campus infrastructure, saves time, increases accountability and gives students a better hostel experience.

11. Future Scope

The Smart PG & Hostel Management System can be significantly enhanced in the following ways:

- **Online Payment Gateway:** integration of Razorpay / UPI payment gateway with automatic receipt and instant status update.
- **Mobile Application:** Android and iOS applications (or a Progressive Web App) for students and wardens using the existing REST API.
- **SMS, E-mail and WhatsApp Alerts:** automatic reminders for fee due dates, leave approval, visitor approval and complaint status changes.
- **QR Code / Biometric Gate Pass:** QR code based verification of leave and visitor passes at the hostel gate and biometric attendance.
- **IoT and Smart Room Features:** smart electricity meters for automatic electricity charge calculation and IoT based door locks.
- **AI and Analytics:** prediction of room demand, detection of frequently failing assets and a chatbot for student queries.
- **Multi-Hostel / Multi-Tenant Support:** one installation serving many hostels or PG owners with separate data and branding.
- **Advanced Reports:** Excel export, graphical analytics for revenue and occupancy and downloadable year-wise reports.
- **Mess Management:** feedback on food, inventory and attendance based mess billing.
- **Security Hardening:** two-factor authentication, CSRF protection for all forms, login attempt limiting and role-wise permission editor.
- **Decoupled React Client:** completing the React single-page client already available in the repository as the main interface.

12. Team Contribution

The project was carried out by **Team Nexus**, a team of five students of B.Tech (Artificial Intelligence & Data Science), Section A, under the guidance of **Er. Ram Babu Buri**, Assistant Professor. Each member owned one primary module and also participated in integration, testing and documentation, so that the code stays modular and every member can explain both his own module and the complete system architecture.

Team Member	Roll No.	Role	Primary Ownership
Dhruvraj Shekhawat	24EARAD045	Team Leader & Backend Lead	Authentication, Spring Boot architecture, backend integration
Divyansh Paliwal	24EARAD047	Frontend, Security & Documentation Lead	Bootstrap 5 UI, sessions, validation, dashboards, report and PPT
Divyanshu Kumar	24EARAD048	Finance & Reporting Developer	Fees, payments, financial logic, reports
Garvit Sharma	24EARAD050	Database & Room Management Developer	MySQL, JPA / Hibernate, rooms, allocation
Harsh Sharma	24EARAD057	Student & Complaint Module Developer	Students, complaints, visitors, leave

Dhruvraj Shekhawat (24EARAD045) - Team Leader & Backend Lead:

- Finalised the project scope, modules and technical architecture and coordinated the abstract, responsibilities and team reviews.
- Created the GitHub repository, Maven / Spring Boot project structure and the commit conventions.
- Prepared the system use case diagram and the login / authentication sequence diagram and documented authentication, session, role and backend requirements in the SRS.

Divyansh Paliwal (24EARAD047) - Frontend, Security & Documentation Lead:

- Finalised the title, objective, target users and problem statement and prepared the abstract draft and documentation structure.
- Designed the login, dashboard and navigation concept and Bootstrap mock-ups; documented the login UI, session flow, role-based access, validation and responsive UI requirements in the SRS.
- Built the responsive Bootstrap login, common navigation, session-aware menus and role dashboards and added BCrypt password hashing support with error messages.

Divyanshu Kumar (24EARAD048) - Finance & Reporting Developer:

- Analysed rent, mess, electricity, maintenance and other fee components and the payment and report requirements; documented fee generation, due dates, payments and report requirements in the SRS.
- Designed the Fee and Payment classes and workflow diagrams and created the fees and payments tables and fee calculation fields.

- Implemented the Fee entity, repository and service with total and pending calculations and the PAID, PENDING, PARTIAL and OVERDUE status logic.

Garvit Sharma (24EARAD050) - Database & Room Management Developer:

- Identified database entities and relationships and analysed room, bed capacity and availability rules.
- Documented room management and allocation / vacating requirements and designed the Room and RoomAllocation classes.
- Created the rooms and room_allocations tables with keys, constraints, indexes and sample data and finalised the ER diagram and database documentation.

Harsh Sharma (24EARAD057) - Student & Complaint Module Developer:

- Identified student profile and management requirements and analysed the complaint, visitor and leave workflows.
- Documented student registration, profile, CRUD, complaint, visitor and leave requirements and designed the Student, Complaint, VisitorRequest and LeaveRequest classes and flows.
- Created the students, complaints, visitor_requests and leave_requests tables with relationships and sample data.

Weekly Evidence Policy:

As per the PBL guidelines the team followed a minimum of three meaningful GitHub commits per member per week, a daily work log with a weekly report, and a branching strategy of main (stable), develop (integration) and feature branches such as feature/room-module and feature/finance-module.

13. References

- Spring Boot Reference Documentation: https://spring.io/projects/spring-boot
- Spring Security Reference: https://docs.spring.io/spring-security/reference/
- Spring Data JPA Reference Documentation: https://spring.io/projects/spring-data-jpa
- Hibernate ORM Documentation: https://hibernate.org/orm/documentation/
- Thymeleaf Documentation: https://www.thymeleaf.org/documentation.html
- Bootstrap 5.3 Documentation: https://getbootstrap.com/docs/5.3/
- MySQL 8.0 Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- JJWT (Java JWT) Library: https://github.com/jwtk/jjwt
- OpenPDF Library: https://github.com/LibrePDF/OpenPDF
- Docker Documentation: https://docs.docker.com/
- Render Documentation: https://render.com/docs
- Aiven for MySQL Documentation: https://aiven.io/docs/products/mysql
- OWASP Top Ten Web Application Security Risks: https://owasp.org/www-project-top-ten/
- Craig Walls, Spring in Action, 6th Edition, Manning Publications, 2022.
- Herbert Schildt, Java: The Complete Reference, McGraw-Hill Education.
- Vlad Mihalcea, High-Performance Java Persistence, 2016.
- Ian Sommerville, Software Engineering, 10th Edition, Pearson Education, 2015.
- Roger S. Pressman, Software Engineering: A Practitioner's Approach, McGraw-Hill Education.
- Ramez Elmasri and Shamkant Navathe, Fundamentals of Database Systems, Pearson Education.
- Grady Booch, James Rumbaugh and Ivar Jacobson, The Unified Modeling Language User Guide, Addison-Wesley.
- GitHub repository of the project: https://github.com/Divyanshp63/smart-hostel-management
- Deployed application: https://smart-hostel-management-9j3u.onrender.com/login
- Online tutorials and technical resources from Baeldung, GeeksforGeeks and Stack Overflow for implementation guidance.

14. Appendices

A. Project Links

Item	Link / Detail
Source Code (GitHub)	https://github.com/Divyanshp63/smart-hostel-management
Live Application (Render)	https://smart-hostel-management-9j3u.onrender.com/login
Technology Stack	Java 17, Spring Boot 3.3.4, Spring MVC, Spring Security 6, JPA / Hibernate 6, Thymeleaf, Bootstrap 5, MySQL 8
User Roles	WARDEN, STUDENT, ACCOUNTANT, COMPLAINT_STAFF
Project Size	Approximately 12,000 lines of Java code and 7,500 lines of Thymeleaf / HTML templates

B. Project Code (Key Extracts)

The following extracts from the source code show the most important logic of the system. They are shortened for the report; the complete code is available in the GitHub repository.

B.1 Role-based Security Configuration (SecurityConfig.java):

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

.authorizeHttpRequests(auth -> auth
    .requestMatchers("/", "/login", "/register", "/css/**", "/js/**").permitAll()
    .requestMatchers("/warden/**", "/api/warden/**").hasRole("WARDEN")
    .requestMatchers("/accountant/**", "/api/accountant/**").hasRole("ACCOUNTANT")
    .requestMatchers("/student/**", "/api/student/**").hasRole("STUDENT")
    .requestMatchers("/complaint-staff/**", "/maintenance/**").hasRole("COMPLAINT_STAFF")
    .anyRequest().authenticated()
)
.formLogin(form -> form
    .loginPage("/login").loginProcessingUrl("/login")
    .usernameParameter("email").passwordParameter("password")
    .successHandler(authSuccessHandler)
    .failureUrl("/login?error=true"))
.logout(logout -> logout
    .logoutUrl("/logout").logoutSuccessUrl("/login?logout=true")
    .invalidateHttpSession(true)
    .deleteCookies("SMART_HOSTEL_SESSION", "JSESSIONID"))
```

B.2 Room Allocation Approval (RoomAllocationServiceImpl.java):

```
@Transactional
public RoomAllocationResponse approveAllocation(Long allocationId, ApprovalRequest request) {
    RoomAllocation allocation = roomAllocationRepository.findById(allocationId)
        .orElseThrow(() -> new ResourceNotFoundException("Room allocation not found"));
    if (allocation.getStatus() != AllocationStatus.PENDING)
        throw new BadRequestException("Only PENDING allocation requests can be approved");

    Student student = allocation.getStudent();
    if (roomAllocationRepository.existsByStudentIdAndStatus(student.getId(), AllocationStatus.ACTIVE))
        throw new BadRequestException("Student already has another ACTIVE room allocation");
```

```
    Room room = allocation.getRoom();
    if (room.getStatus() == RoomStatus.UNDER_MAINTENANCE)
        throw new BadRequestException("Room is currently under maintenance");
    if (room.getAvailableBeds() <= 0)
        throw new BadRequestException("Room has no available beds");

    room.setOccupied(room.getOccupied() + 1);          // increase occupancy
    room.updateStatusBasedOnOccupancy();               // AVAILABLE or FULL
    roomRepository.save(room);

    allocation.setStatus(AllocationStatus.ACTIVE);
    allocation.setBedNumber(request.getBedNumber().trim().toUpperCase());
    allocation.setStartDate(LocalDate.now());
    return mapToResponse(roomAllocationRepository.save(allocation));
}
```

B.3 Payment Processing (PaymentServiceImpl.java):

```
// Ownership: a student can pay only his own invoice
if (currentUser.getRole() == Role.STUDENT &&
    !fee.getStudent().getUser().getId().equals(currentUser.getId()))
    throw new UnauthorizedException("You are not authorized to pay another student's invoice");

if (fee.getStatus() == FeeStatus.PAID || fee.getPendingAmount().compareTo(BigDecimal.ZERO) <= 0)
    throw new BadRequestException("Fee invoice is already fully PAID");
if (request.getAmount().compareTo(fee.getPendingAmount()) > 0)
    throw new BadRequestException("Payment amount cannot exceed pending fee balance");

String transactionId = "TXN-" + System.currentTimeMillis() + "-" +
        UUID.randomUUID().toString().substring(0, 6).toUpperCase();

fee.setPaidAmount(fee.getPaidAmount().add(request.getAmount()));
fee.recalculateTotalsAndStatus();                      // PARTIAL / PAID
Fee updatedFee = feeRepository.save(fee);

Payment payment = Payment.builder()
    .fee(updatedFee).student(updatedFee.getStudent())
    .transactionId(transactionId).amount(request.getAmount())
    .paymentMethod(request.getPaymentMethod())
    .paymentStatus(PaymentStatus.SUCCESS)
    .paymentDate(LocalDateTime.now()).build();
paymentRepository.save(payment);
```

B.4 Complaint Creation with Automatic Routing and SLA (ComplaintServiceImpl.java):

```
ComplaintPriority priority = request.getPriority() != null
        ? request.getPriority() : ComplaintPriority.MEDIUM;
LocalDateTime slaDeadline = LocalDateTime.now().plusHours(priority.getSlaHours());

Complaint complaint = Complaint.builder()
    .student(student).room(room)
    .title(request.getTitle().trim())
    .description(request.getDescription().trim())
    .category(request.getCategory())
    .priority(priority)
    .status(ComplaintStatus.SUBMITTED)                  // automatic routing
    .assignedDepartment(ComplaintCategory.MAINTENANCE)
    .slaDeadline(slaDeadline)
    .build();
Complaint saved = complaintRepository.save(complaint);
auditLogService.log(userId, userName, "STUDENT", "COMPLAINT_SUBMITTED", "COMPLAINT", ...);
notificationService.notifyMaintenance("New Complaint Submitted", ..., saved.getId());
```

C. REST API Catalog (Summary)

Area	Endpoint	Purpose
Authentication	POST /api/auth/login, /register; GET /api/auth/me	Login, register and current user
Dashboard	GET /api/dashboard/admin, /student	Role dashboards
Rooms	GET/POST /api/rooms; GET /api/rooms/available	Room CRUD and availability
Allocations	POST /api/allocations/request, approve, reject, vacate	Allocation workflow
Fees	GET /api/fees; POST /api/fees/bulk; GET /api/fees/my	Invoices and bulk generation
Payments	POST /api/payments; GET /api/payments/transaction/{id}	Payment and lookup
Complaints	POST /api/complaints; PUT /api/complaints/{id}/status	Ticket lifecycle
Visitors / Leaves	POST /api/visitors/request, approve; POST /api/leaves/apply, approve	Passes and approvals
Mess / Notices	GET /api/mess-menu/today, weekly; GET/POST /api/notices	Menu and announcements
Reports	GET /api/reports/fee-receipt/{id}, /financial/monthly, /occupancy	PDF documents